

Multi-Fidelity Field Prediction (MFFP)

Detailed answers to **QUESTIONS.md** — model performance, mechanisms, a path to high-frequency solutions, and the inspiration behind every model family.

Generated 2026-06-27. Numbers transcribed from *model/README.md* (leaderboard, 30 families × 15 datasets; per-sample relative-L2 with 95% bootstrap CIs, reported as the median per dataset; shared geometry-general FNO backbone, seed 42, 2500 epochs) and *baselines/paper_baselines.json* (fixed published reference nRMSE). Provenance from each family's *manifest.json* / *INSPIRATION.md*. Source of truth for regeneration: *results/bench_metrics.csv*, *results/bench_elo.csv* on the cluster.

Questions being answered

- **Q1.** How well are the models performing — e.g. how can I visualize the performance difference between the best model and a *perfect* model?
- **Q2.** How does each of the models work? How do we optimize them, or create a new one, to be better at multi-fidelity field prediction — especially for solutions with *high-frequency* content?
- **Plus:** what inspiration (source paper) was each model family created from?

1. How well are the models performing?

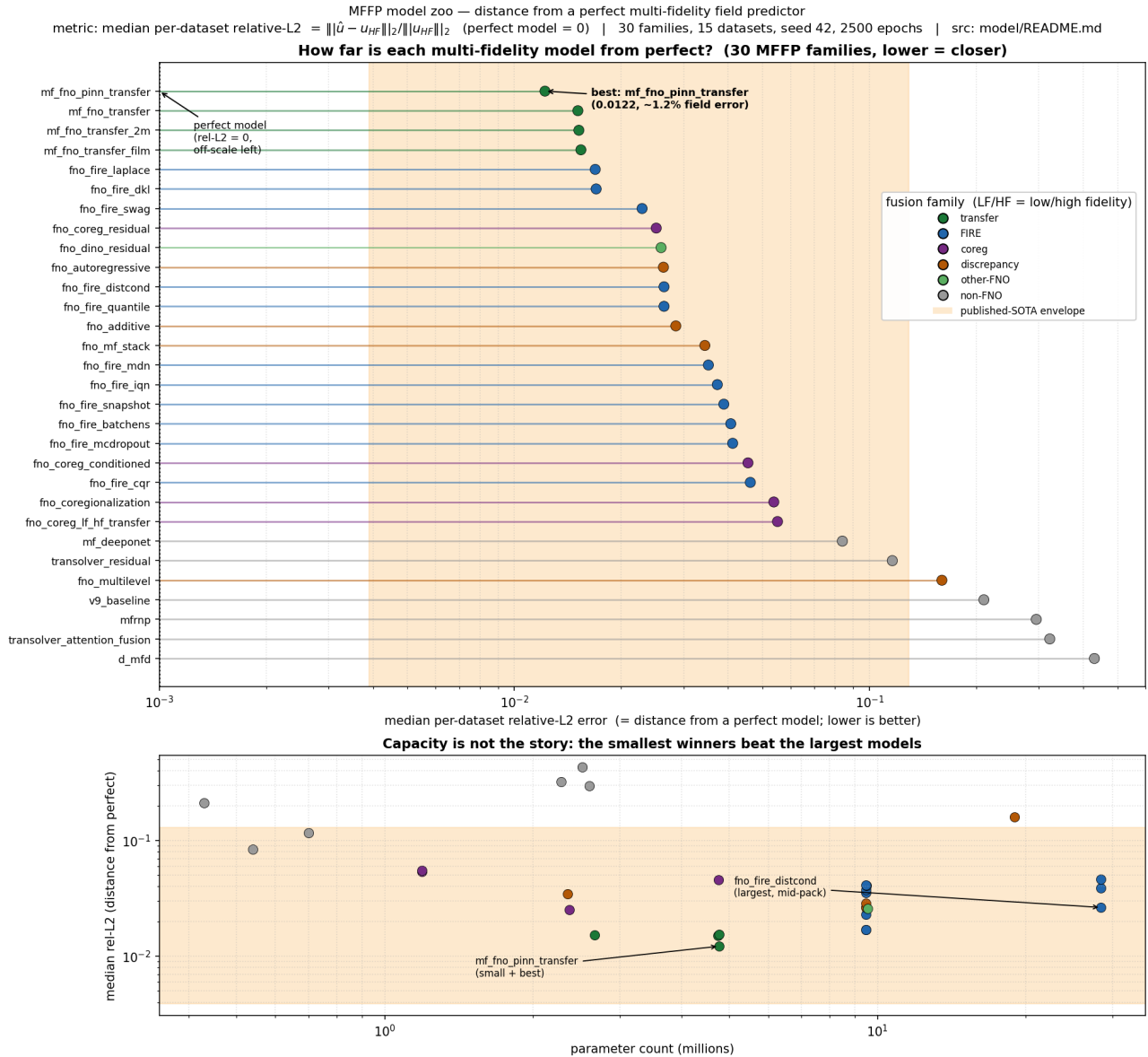
A perfect multi-fidelity model reconstructs the high-fidelity (HF) field exactly, so its relative- L_2 error is 0.

That is the key to the question: the error metric *is* the distance from perfect. “How well are we doing” reduces to “how close to zero is the bar, and where does published SOTA sit on the same axis.” Relative- $L_2 = \|\hat{u} - u_{\text{HF}}\|_2 / \|u_{\text{HF}}\|_2$, where $\hat{u}$ is the prediction.

- **Lowest median error:** `mf_fno_pinn_transfer` at median per-dataset relative- $L_2 = 0.0122$ (~1.2% field error).
Highest Elo (the leaderboard's primary rank): `mf_fno_transfer_film` at a similarly low median 0.0154. Either way the HF field is reconstructed to ~98.5% relative accuracy on the median dataset — an intuitive reading of the error, not a separate metric.
- **vs. published SOTA** (paper_baselines.json): mixed and dataset-dependent. The tracked per-dataset numbers are the in-house **v9** reference: it beats the paper on `heat_local`'s in-distribution test splits (~0.001 vs 0.004–0.005) but *trails* on `poisson_local` (0.014–0.023 vs paper 0.005–0.008); `ifc_heat/ifc_poisson` carry paper-only targets (0.074 / 0.036). The champion's per-dataset numbers live in the cluster bench CSVs, so SOTA is confirmed on some datasets, not uniformly.
- **Capacity is not the story.** The two winners are small (4.77M params), while the largest models span the whole table — the 28.4M FIRE variants land at #3, #4 and #14, and the 18.96M `fno_multilevel` is #25. Size doesn't predict rank; the multi-fidelity *fusion mechanism* does.

1.1 Visualizing “best vs. perfect” — the figure

The figure below is the cross-model view. Top panel: every family as a lollipop on a **log-x** axis (errors span 0.012→0.43, so log linearizes them); the perfect model (error = 0) is marked off-scale at the left edge, and the shaded band is the published-SOTA envelope across datasets. Bottom panel: error vs. parameter count (log-log), making the “capacity is not the story” point explicit. It re-renders in seconds (`leaderboard_distance_from_perfect.py`).



Reading it: green = transfer-learning FNOs (the winners), blue = FIRE uncertainty-conditioned FNOs (a tight high-performing cluster), purple = coregionalization, orange = classical correction rules, grey = non-FNO backbones (DeepONet / neural-process / Bayesian / Transformer), which trail.

1.2 The visualization to add next (most intuitive)

The cross-model figure compares *models*. The most intuitive “best vs. perfect” view is per-sample and field-level — it needs a model run on cluster data, so it is specified here rather than rendered:

- Four panels at a shared color scale: **LF input** (shown blocky at native resolution) · **best-model prediction** · **HF truth** (= the “perfect” target) · **error map** (prediction – truth, diverging colormap symmetric about 0).
- A fifth panel: the **radial power spectrum** of prediction vs. truth on log-log — this shows *at which spatial frequencies* the error lives, which is the direct bridge to Q2 (high-frequency error).
- Follow the field-one-pager recipe: a native-resolution field row, descriptive “Error between ...” panel titles, a mathtext PDE + condition-vector header, and an annotated radial-spectrum slope. (The leaderboard figure shipped with this report — `leaderboard_distance_from_perfect.py` — already follows these cold-reader conventions and can serve as the style template.)

2. How does each of the models work?

Most families (24 of 30) share the **same Fourier Neural Operator (FNO) backbone**; they differ only in **where the multi-fidelity fusion happens** (the 6 non-FNO backbones are the last row). That single axis organizes the whole zoo:

Where fusion lives	Families	Core idea
Training schedule	mf_fno_transfer, _film, _2m; mf_fno_pinn_transfer	Pretrain one FNO on abundant LF data, then fine-tune on scarce HF. MF lives in the schedule, not extra architecture. The winners.
Conditioning layer	mf_fno_transfer_film; fno_coreg_conditioned; fno_coregionalization	Inject the condition/fidelity via FiLM affine (gamma,beta) or a coregionalization basis $B(m)$, instead of channel concatenation.
Correction structure	fno_additive, fno_autoregressive, fno_multilevel, fno_mf_stack	Explicit HF = $(\rho \cdot)$ LF_pred + delta with extra FNOs — the classical surrogate-correction rules, FNO-ized.
Distributional signal	fno_fire_* (11 families)	Condition the residual delta on the LF <i>predictive uncertainty</i> fields [mu, sigma, q10, q50, q90]; variants only swap the uncertainty source.
Different backbone	transolver_*, mf_deeponet, mfrnp, d_mfd, v9_baseline	Same MF goal, non-FNO operator: physics-attention, branch-trunk DeepONet, neural process, Bayesian active learning, gated Transformer.

How each archetype fuses, in one line each:

- **Transfer (schedule).** A single FNO is pretrained on cheap LF samples, then the same weights are fine-tuned on the few HF samples. **mf_fno_transfer_film** adds per-block FiLM (gamma,beta from the condition vector) instead of concatenation — the sole change that lifts it to #1, pinning the win on the *conditioning mechanism*. **mf_fno_pinn_transfer** adds an exact Poisson PDE-residual loss on top.
- **Coregionalization.** An IFC-style head modulates a low-rank latent grid by a continuous fidelity index m via a learned basis $B(m) = \text{MLP}([m, m^2])$; **fno_coreg_conditioned** moves that modulation into FiLM-via-LayerNorm.
- **Correction rules.** **additive:** HF = LF_pred + delta. **autoregressive:** HF = $\rho \cdot$ LF_pred + delta with a learned gain ρ (Kennedy & O'Hagan). **multilevel:** a telescoping cascade $\text{pred}_k = \text{pred}_{\{k-1\}} + \text{delta}_k$ (recursive co-kriging). **mf_stack:** per-fidelity FNOs + MFRNP-style decoder-in-the-aggregation.
- **FIRE.** An uncertainty estimator turns the LF model into per-pixel distribution fields [mu, sigma, q10, q50, q90]; a residual FNO conditions on those and predicts HF = $\mu_{\text{LF}} + \text{delta}$. All 11 follow this recipe (9 share the `models/_common/fire_core+fire_methods` code; `distcond` and `mcdropout` are implemented separately) and only change *how* the LF distribution is obtained — a clean ablation of which uncertainty estimate helps most.

Empirical lessons (from the leaderboard): (1) schedule + the right conditioning beats architectural elaboration; (2) known physics adds a little more; (3) LF uncertainty is a useful conditioning signal (FIRE clusters high), but the cheap MC-dropout shortcut and DINO vision features are honest negatives; (4) classic correction rules and external NP/DeepONet/Bayesian baselines trail.

3. Optimizing — or building a new family — for high-frequency solutions

3.1 Why high frequency is the structural weak spot

The backbone's spectral convolution (*models/_common/fire_core.py*, `SpectralConv2d`) does: `rfft2` → keep only the low-frequency bands (the `[:mh,:mw]` and `[-mh:,:mw]` corners) → linear transform → `irfft2`; **every higher mode is zeroed**. Each block adds a parallel pointwise 1×1 conv (`self.w`), but being pointwise it cannot *synthesize* spatial frequencies the spectral path discarded — it only passes through frequencies already present in its input. So high-frequency content above the cutoff is structurally hard for the backbone to produce: the FNO's well-known spectral bias, here made hard by truncation.

The cruel part for multi-fidelity: the LF→HF discrepancy **delta** — the entire point of the task — is dominated by *high-frequency detail* (sharp gradients, fronts, boundary layers) that coarse LF simulations miss. So the signal MF must recover is precisely what the backbone is worst at, and every FNO family inherits this ceiling. To do better on high-frequency solutions you must attack the *spectral truncation*, not just the fusion mechanism.

3.2 Concrete, ablatable directions (ranked)

Each fits the existing $HF = \mu_{LF} + \delta$ contract and is a clean, independently-testable knob:

#	Direction	What it does / why it targets high frequency
1	Wavenumber-weighted spectral loss (cheapest, do first)	Plain $\text{rel-}L_2$ is dominated by low-mode energy (Parseval), so the optimizer barely penalizes high-frequency error and the model ignores it. Add a loss on $ \text{FFT}(\hat{u}) - \text{FFT}(u) $ weighted by wavenumber k (or a high-pass residual loss). Backbone-agnostic; directly attacks the metric blind spot.
2	High-frequency residual branch	Keep the cheap low-mode FNO for the bulk; add a parallel branch for the high-mode band — a second <code>SpectralConv</code> on the high-mode annulus, or a local conv / U-Net branch that captures fine spatial detail the truncated path drops. Let δ be predicted by this branch.
3	Fourier-feature input lifting / sinusoidal activations	Per the spectral-bias literature, lifting inputs with high-frequency Fourier features (or SIREN-style sinusoidal activations) lets the non-spectral pointwise path represent fine detail. Cheap and orthogonal to the fusion mechanism.
4	Wavelet / multiwavelet operator backbone	Replace/augment the global FFT with a wavelet transform (WNO) or multiwavelet operator. Wavelets localize in space AND frequency, so sharp local high-frequency features are captured with far fewer coefficients than a globally-truncated Fourier basis — the principled operator-learning answer to high frequency + locality.
5	High-pass residual target	Since LF already nails the low frequencies, train δ on the high-pass-filtered ($HF - LF_{\text{pred}}$), spending all capacity on the hard high-frequency correction. Composes cleanly with FIRE's uncertainty conditioning.
6	Generalize the PDE residual (extends <code>mf_fno_pinn_transfer</code> , #2)	PDE residuals involve derivatives, and differentiation multiplies mode k by k — so a PDE-residual loss amplifies high-frequency error, penalizing it far more than data MSE. Generalize the exact-Poisson approach to spectral-derivative residuals on more datasets.

3.3 Recommended new family: a *spectral-residual transfer-FNO*

Start from the current champion (the FiLM transfer-FNO) and stack three independent, ablatable knobs: **(1)** the wavenumber-weighted spectral loss, **(2)** a high-mode residual branch, and **(3)** Fourier-feature input lifting — optionally **(4)** a high-pass residual target. It is incremental over the proven winner, squarely targets the high-frequency ceiling, and every piece slots into `models/<new_family>/` under the existing contract (`manifest.json` + `smoke_eval.py` + `INSPIRATION.md`, checkpoint-resume mandatory). Because creating a new family is creative work, the next step would run it through a brainstorming pass to pin the exact ablation matrix before implementing.

4. Inspiration: what each model family was created from

Per the repo contract, new in-repo families are implemented *from scratch from their source paper(s)*, while the external-SOTA entries (mfrnp, mf_deepnet, d_mfd) are faithful wrappers of published upstream code. Provenance is recorded in each family's `manifest.json` (`inspired_by / paper`) and, for newer families, an `INSPIRATION.md` with bibtex keys. The shared backbone for 24 of 30 families is the **Fourier Neural Operator** (li2020fno). The table maps each family to its source idea and what was taken; full references follow.

Family	Source paper(s)	What was taken / how it fuses
Transfer-learning FNO — the winners		
mf_fno_transfer	Lyu et al. 2023 (MF-FNO transfer); Li et al. 2021 (FNO)	Pretrain one FNO on LF, fine-tune on HF. The canonical “what transfer alone buys you.”
mf_fno_transfer_film	+ Perez et al. 2018 (FiLM)	Same transfer-FNO, condition vector injected by per-block FiLM affine (γ, β) instead of concatenation. Best Elo overall.
mf_fno_transfer_2m	Lyu et al. 2023 (MF-FNO transfer); Li et al. 2021 (FNO)	Parameter-matched control of mf_fno_transfer (width 64→48, ~2.67M) isolating the transfer mechanism from raw capacity.
mf_fno_pinn_transfer	+ Raissi et al. 2019 (PINN)	FiLM-transfer + an exact Poisson PDE-residual loss during HF fine-tuning (source f recovered in closed form). Lowest median error.
Coregionalization FNO — continuous-fidelity head		
fno_coregionalization	Li et al. 2022 (IFC); Li et al. 2021 (FNO)	IFC continuous-fidelity head $f(x, m) = \sum_k B_k(m) h_k(x)$, $B(m) = \text{MLP}([m, m^2])$, $K=10$ latent, per-fidelity output scaling.
fno_coreg_conditioned	+ Beggs et al. 2025 (PDE FiLM-via-LayerNorm); Herde et al. 2024 (Poseidon); Lyu 2023 (transfer)	Replaces the K -dim basis with FiLM-via-LayerNorm on m (GroupNorm + per-channel affine from an MLP on $[m, m^2]$).
fno_coreg_lf_hf_transfer	Li 2022 (IFC) + Lyu 2023 (transfer)	Coregionalization FNO trained with an explicit LF→HF transfer schedule.
fno_coreg_residual	Li 2022 (IFC) + Niu 2024 (MFRNP); cf. Xing 2021 (ResPCA)	Hybrid: 4 per-fidelity FNOs + MFRNP decoder-in-the-aggregation + a zero-init continuous- m coregionalization residual head on the HF latent.
Discrepancy / multi-level correction FNO		
fno_additive	Peherstorfer et al. 2018 (MF survey)	Additive correction $HF = LF_{\text{pred}} + \delta$. Frozen FNO_LF on LF data; FNO_delta fits the HF residual. Pure 2-fidelity additive rule.
fno_autoregressive	Kennedy & O'Hagan 2000 (AR1 co-kriging)	Multiplicative/AR correction $HF = \rho \cdot LF_{\text{pred}} + \delta$ with a learned scalar gain ρ .
fno_multilevel	Le Gratiet & Garnier 2014 (recursive co-kriging)	Recursive telescoping cascade $\text{pred}_k = \text{pred}_{[k-1]} + \delta_k$, one FNO per fidelity level. 18.96M params (largest of the correction families; the 28.4M FIRE variants are larger overall).
fno_mf_stack	Niu et al. 2024 (MFRNP); Li 2021 (FNO)	4 small per-fidelity FNOs, MFRNP-style residual stacking: decoder-in-the-aggregation, HF FNO predicts δ on the upsampled-aggregate LF prediction.
FIRE distribution-conditioned FNO — uncertainty-source ablation (all: Yu, Sung & Ahmed 2026, arXiv:2601.22371)		
fno_fire_distcond	+ Lakshminarayanan et al. 2017 (deep ensembles)	LF uncertainty from a deep ensemble of LF-FNOs. The strongest FIRE variant (#3).
fno_fire_snapshot	+ Huang et al. 2017 (snapshot ensembles)	Snapshot ensemble (cyclic-LR, 5 snapshots, one training run).
fno_fire_batchens	+ Wen et al. 2020 (BatchEnsemble)	Multi-head / batch ensemble (shared trunk, 5 heads).
fno_fire_mdn	+ Bishop 1994 (Mixture Density Networks)	Mixture density network, 3 Gaussians per pixel.

Family	Source paper(s)	What was taken / how it fuses
fno_fire_iqn	+ Dabney et al. 2018 (IQN)	Implicit quantile network (tau-conditioned quantiles).
fno_fire_quantile	+ Koenker & Bassett 1978 (quantile regression)	Direct quantile regression (pinball loss, q10/q50/q90 heads).
fno_fire_laplace	+ MacKay 1992 / Daxberger et al. 2021 (Laplace)	Last-layer Laplace (Gaussian weight posterior via GGN).
fno_fire_swag	+ Maddox et al. 2019 (SWAG)	SWAG: diagonal Gaussian over the SGD-tail weights, 10 samples.
fno_fire_cqr	+ Romano et al. 2019 (Conformalized Quantile Regression)	Snapshot ensemble + conformal calibration (coverage-guaranteed percentiles).
fno_fire_dkl	+ Wilson et al. 2016 (Deep Kernel Learning)	Deep-kernel last-layer GP (Bayesian linear layer on FNO features).
fno_fire_mcdropout	+ Gal & Ghahramani 2016 (MC-dropout)	Cheap single-model FIRE: one dropout LF-FNO, MC-dropout passes for the distribution fields (honest negative).
Vision-foundation-feature & Transformer / external SOTA		
fno_dino_residual	Oquab et al. 2023 (DINOv2); Simeoni et al. 2025 (DINOv3, swap-ready)	Additive FNO correction conditioned on a frozen DINOv2 768-d embedding of the LF field as an image (honest negative).
transolver_residual	Wu et al. 2024 (Transolver); Li et al. 2022 (IFC); Niu et al. 2024 (MFRNP)	Physics-attention (slice-attention) backbone trained as a hard MF residual $HF = LF_interp + transolver_delta$.
transolver_attention_fusion	Wu et al. 2024 (Transolver); Li et al. 2024 (OFormer); Vaswani et al. 2017 (attention)	Transolver with iterated cross-fidelity attention as the fusion (no explicit residual or gate).
mfrnp	Niu et al. 2024 (MFRNP, ICML)	Faithful wrapper of Multi-Fidelity Residual Neural Processes.
mf_deeponet	Lu et al. 2022 (multifidelity DeepONet)	Faithful wrapper of branch-trunk multifidelity DeepONet.
d_mfd	Wu et al. 2023 (Disentangled MF Deep Bayesian AL)	Faithful wrapper of the disentangled multi-fidelity Bayesian model.
v9_baseline	in-house MFTransolver_v9 (frozen)	Point-wise gated fusion over multiple LF streams. The reference baseline to beat.

References

Papers cited in the families' manifests / INSPIRATION.md (and, for the high-frequency directions in §3, suggested supporting literature, flagged).

- **li2020fno** — Li, Z. et al. (2021). Fourier Neural Operator for Parametric PDEs. ICLR. arXiv:2010.08895. (*shared backbone*)
- **lyu2023mffno** — Lyu, Y. et al. (2023). Transfer-learning multi-fidelity FNO (MF-FNO). (*bibtex key as cited in repo*)
- **perez2018film** — Perez, E. et al. (2018). FiLM: Visual Reasoning with a General Conditioning Layer. AAAI.
- **raissi2019pinn** — Raissi, M., Perdikaris, P., Karniadakis, G.E. (2019). Physics-Informed Neural Networks. J. Comput. Phys.
- **li2022ifc** — Li, S., Wang, Z., Kirby, R.M., Zhe, S. (2022). Infinite-Fidelity Coregionalization for Physical Simulation. NeurIPS.
- **herde2024poseidon** — Herde, M. et al. (2024). Poseidon: Efficient Foundation Models for PDEs. NeurIPS.
- **beggs2025pdecond** — Beggs et al. (2025). FiLM-via-LayerNorm conditioning for parametric-PDE surrogates. (*bibtex key as cited in repo*)
- **niu2024mfrnp** — Niu, R. et al. (2024). Multi-Fidelity Residual Neural Processes for Scalable Surrogate Modeling. ICML.
- **xing2020deepcoreg** — Xing, W.W. et al. (2021). Deep coregionalization (ResPCA). J. Comput. Phys. 428.
- **kennedy2000ar** — Kennedy, M.C., O'Hagan, A. (2000). Predicting the output of a complex code when fast approximations are available. Biometrika.
- **legatiet2014** — Le Gratiet, L., Garnier, J. (2014). Recursive co-kriging / multi-level surrogate modeling.
- **peherstorfer2018survey** — Peherstorfer, B., Willcox, K., Gunzburger, M. (2018). Survey of multifidelity methods. SIAM Review.
- **yu2026fire** — Yu, Sung, Ahmed (2026). FIRE: Multi-fidelity Regression with Distribution-conditioned In-context Learning using Tabular Foundation Models. arXiv:2601.22371.
- **lakshminarayanan2017** — Lakshminarayanan, B. et al. (2017). Simple and Scalable Predictive Uncertainty via Deep Ensembles. NeurIPS.
- **huang2017snapshot** — Huang, G. et al. (2017). Snapshot Ensembles. ICLR. arXiv:1704.00109.
- **wen2020batchensemble** — Wen, Y. et al. (2020). BatchEnsemble. ICLR. arXiv:2002.06715.
- **bishop1994mdn** — Bishop, C.M. (1994). Mixture Density Networks.
- **dabney2018iqn** — Dabney, W. et al. (2018). Implicit Quantile Networks for Distributional RL. ICML. arXiv:1806.06923.
- **koenker1978** — Koenker, R., Bassett, G. (1978). Regression Quantiles. Econometrica.
- **daxberger2021laplace** — Daxberger, E. et al. (2021). Laplace Redux. NeurIPS. arXiv:2106.14806 (cf. MacKay 1992).
- **maddox2019swag** — Maddox, W. et al. (2019). A Simple Baseline for Bayesian Uncertainty (SWAG). NeurIPS. arXiv:1902.02476.
- **romano2019cqr** — Romano, Y. et al. (2019). Conformalized Quantile Regression. NeurIPS. arXiv:1905.03222.
- **wilson2016dkl** — Wilson, A.G. et al. (2016). Deep Kernel Learning. AISTATS. arXiv:1511.02222.
- **gal2016dropout** — Gal, Y., Ghahramani, Z. (2016). Dropout as a Bayesian Approximation. ICML. arXiv:1506.02142.
- **oquab2023dinov2** — Oquab, M. et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193. (with DINOv3: Simeoni, M. et al. 2025, arXiv:2508.10104).
- **wu2024transolver** — Wu, H. et al. (2024). Transolver: A Fast Transformer Solver for PDEs on General Geometries. ICML.
- **li2024oforner** — Li, Z. et al. (2024). OFormer: Operator Transformer for PDEs. (*bibtex key as cited in repo*)
- **vaswani2017attention** — Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- **lu2022deeponet** — Lu, L. et al. (2022). Multifidelity DeepONet. Phys. Rev. Research.
- **wu2023dmfd** — Wu, D. et al. (2023). Disentangled Multi-Fidelity Deep Bayesian Active Learning. ICML.
- [*§3 suggested*] **tancik2020** — Tancik, M. et al. (2020). Fourier Features Let Networks Learn High-Frequency Functions. NeurIPS.
- [*§3 suggested*] **sitzmann2020siren** — Sitzmann, V. et al. (2020). Implicit Neural Representations with Periodic Activations (SIREN). NeurIPS.
- [*§3 suggested*] **tripura2023wno** — Tripura, T., Chakraborty, S. (2023). Wavelet Neural Operator. CAME.

- *[§3 suggested]* **gupta2021mwt** — Gupta, G. et al. (2021). Multiwavelet-based Operator Learning for Differential Equations. NeurIPS.